26/02/20

POLYMORPHISM

It is the ability of the members to exhibit more than one forms is known as Polymorphism.

In java, we can achieve polymorphism in two ways:

1) Compile time polymorphism [static binding]
2) Run time polymorphism [Non-static binding]

COMPILE TIME POLYMORPHISM:-

If which Implementation ~~form~~ of the member has to be used is decided by the compiler during compilation, then we call it as Compile time polymorphism.

In java, we have following compile time polymorphisms:

1) Method Overloading
2) Constructor Overloading
3) Shadowing
   - Variable shadowing
   - Method shadowing.

METHOD OVERLOADING:

A class having methods with same name but different formal arguments is known as Method overloading.

Method overloading is an example of compile time polymorphism as the compiler decides which implementation to be executed based on the actual argument passed. Hence it is known as Compile Time polymorphism.

CONSTRUCTOR OVERLOADING :-

A class having constructors with same name but different formal arguments is known as Constructor Overloading.

Constructor Overloading is an example of compile time polymorphism as the compiler decides which implementation to be executed based on the actual arguments passed. Hence it is known as Compile Time polymorphism.

SHADOWING :

Shadowing is nothing but having members with same name or signature within same class virtually.

We have following types of shadowing:-

1) Variable shadowing.

2) Method shadowing

VARIABLE SHADOWING:-

If both super class & sub class are having "either static or non-static variable" with the same name then it is known as variable shadowing

It is the compiler who decides which variable to be used. based on :

1) Place of Invocation / Place of usage.

2) Type of Reference variable created.

If the type of Rereference variable is of sub-class, sub class member will be used, if the type of reference is of super class then super class member will be used. It doesn't depend on the type of object created.

Eg:

```
class A
{
  int i=10;
}
class B extends A
{
  int i=20;
}
class Driver1
{
  public static void main(String[] args)
    {
      A obj= new B();
      System.out.println(obj.i); //10
      B obj1= new B();
      System.out.println(obj1.i); //20
    }
}
```

METHOD SHADOWING :

If both super class & sub class are having methods with same signature then we call it as method shadowing.

Rules of Method shadowing :

1) There should be ~~is-a~~ is-A relationship between classes.

2) We should have "static Methods" with same signature.

3) The return types of both the methods must be same.

4) The subclass access modifier must always have either similar visibility or higher visibility compared to super class Method. [private < default < protected < public]

NOTE: Method shadowing is also an Ex of compile time polymorphism, as the compiler decides which member should be executed during compilation, based on:

1) Place of Invocation / Usage
2) Type of Reference variable created.

Eg:
```
class A
{
 static void test()
  {
   System.out.println("From-A");
  }
}
class B extends A
 {
   static void test()
    {
      System.out.println("From-B");
    }
 }
class Driver
 {
   public static void main(String[] args)
    {
      B obj = new B();
```

```
        obj.test();  // from B
        A obj1 = new B();
        obj1.test(); // from A
    }
}
```

Eg:
```
class Driver2
{
    public static void main(String[] args)
    {
        A obj = new B();
        ((B)obj).test; // from B.
        System.out.println
    }
}
```

▶ RUN-TIME POLYMORPHISM:-

* It is also known as Non-static binding.
* In the Run-time polymorphism, the binding between method call & Method Implementation to be executed happens during Run time. Hence it is known as Run-time polymorphism.

NOTE: In Run-time polymorphism, ~~whether~~ whatever. compiler will see ~~it~~ may no get executed.

We can achieve Run-time polymorphism with the help of METHOD OVERRIDING.

METHOD OVERRIDING: The process of providing new Implementation to the super class methods from sub class is known as Method overriding.

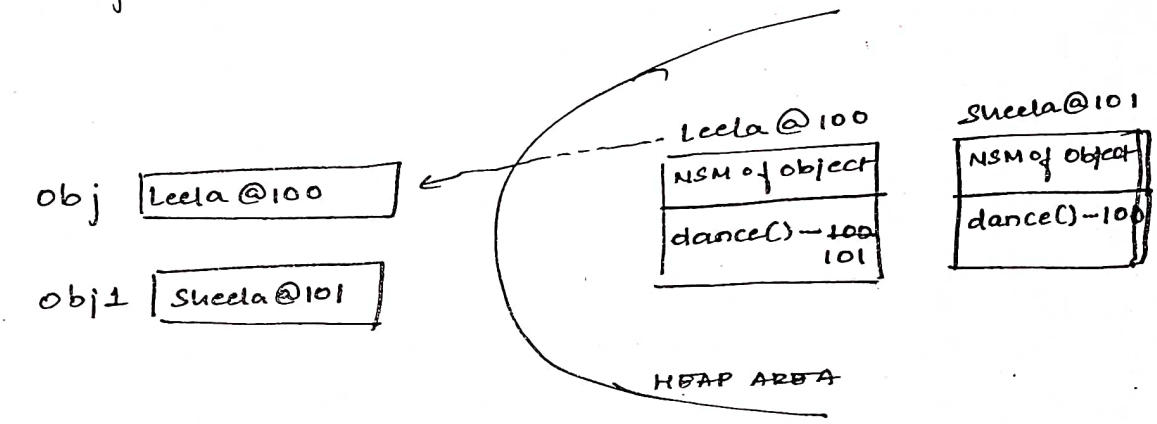
NOTE: Rules of Method Overriding is just similar to rules of Method shadowing, except for having "Non-static methods" instead of having static methods.

Method over-riding is an example for Run-time polymorphism since which implementation to be executed is decided during Run-time based on type of Object created.

NOTE: Static methods cannot be overridden.

Eg:
```
class Sheela
{
 public void dance()
  {
   System.out.println("Bharathanatyam");
  }
}
class Leela
{
 public void dance()
 {
   System.out.println("Hiphop");
 }
}
```

```
class Driver1
{
  public static void main(String[] args)
  {
    Sheela obj = new Leela();
    obj.dance(); // hiphop
    Sheela obj1 = new Sheela();
    obj1.dance();
  }
}
```



The super class address will be updated (overriden) by sub class address.